Intermediate (build the gate) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `release_gate_practice.py`: "test before you release" is exactly the kind of habit people forget under real deadline pressure. A release gate turns that habit into a rule the code itself enforces: no passing eval score means no update to what's actually live, no matter who's asking or how urgently. **If not:** the Build Task's `deploy()` would be the very first gate you ever wrote, guarding Project 5's real prompts with no smaller version to trust it against.

All examples below assume `versions.json` already has `v1` (score 0.6) and `v2` (score 1.0) saved, the same way the `version_log` exercise builds it, and reuse the `run_eval_suite` stub from that exercise.

Solution

Story: a missing version name should raise a clear, named error message, not a confusing crash deep in unrelated code, and the actual pass-score bar is a real team decision that should never silently fall back to some hardcoded default. **If not:** a simple typo in a version name would crash with an opaque `NoneType` error, and a lost `.env` line could quietly lower the release bar with nobody noticing.

```
# release_gate_practice.py
import json
import logging
import os
from dotenv import load_dotenv

logger = logging.getLogger(__name__)
logger.setLevel(logging.INFO)
logger.addHandler(logging.StreamHandler())

class MissingThresholdError(Exception):
    pass

def load_threshold() -> float:
    load_dotenv()
    raw = os.getenv("RELEASE_THRESHOLD")
    if raw is None or raw == "":
        # why: the release bar is a team decision — never guess it
```

```
        raise MissingThresholdError(
            "RELEASE_THRESHOLD is not set in .env — "
            "the gate will not guess its own bar"
        )
    return float(raw)

def load_versions(path: str) -> list[dict]:
    with open(path) as f:
        return json.load(f)

def find_version(versions: list[dict], name: str) -> dict | None:
    for v in versions:
        if v["version_name"] == name:
            return v
    return None

def run_eval_suite(prompt_text: str) -> dict:
    if "politely" in prompt_text:
        return {"score": 1.0}
    return {"score": 0.6}

def deploy(version_name: str, threshold: float) -> None:
    versions = load_versions("versions.json")
    target = find_version(versions, version_name)
    if target is None:
        logger.error(f"No such version: {version_name}")
        return

    result = run_eval_suite(target["prompt_text"])
    if result["score"] >= threshold:
        with open("live_version.txt", "w") as f:
            f.write(version_name)
        score = result["score"]
        logger.info(f"{version_name} deployed — score {score} >= {threshold}")
    else:
        score = result["score"]
        logger.warning(
            f"{version_name} NOT deployed — score {score} < {threshold}"
        )

def get_live_version() -> str | None:
    try:
        with open("live_version.txt") as f:
            return f.read().strip()
    except FileNotFoundError:
        return None
```

```
threshold = load_threshold()
deploy("v2", threshold)
deploy("v1", threshold)
deploy("v9", threshold)
print("currently live:", get_live_version())
```

Expected output, with `RELEASE_THRESHOLD=0.9` in `.env`:

```
v2 deployed — score 1.0 >= 0.9
v1 NOT deployed — score 0.6 < 0.9
No such version: v9
currently live: v2
```

(the first three lines are log lines — this handler has no formatter, so only the message shows)

And with `RELEASE_THRESHOLD` missing from `.env`, nothing is deployed at all. The last line of the traceback:

```
__main__.MissingThresholdError: RELEASE_THRESHOLD is not set in .env — the gate
will not guess its own bar
```

Revision from Doc01 (Intermediate): two habits come back. First, `load_dotenv()` plus Doc01's "required value is missing → raise a named error" rule. Basic used a default of `0.9`; that is fine for a practice script, but the release bar is a real team decision. If `.env` loses that line, a quiet default could let a weaker version go live without anyone choosing that. So the gate refuses to start. Second, a named logger with each outcome at its own level: `INFO` for a deploy, `WARNING` for a refused deploy (safe — nothing changed), and `ERROR` for `v9`, because that one operation failed completely, while the program itself can go on.